



UADE

Interfaces

Paradigmas de Programación



PROBLEMA

Objetivo: quiero poder identificar a todos los objetos dañables y poder acceder a sus campos públicos. Algunos de estos son la clase *Player*, *TankEnemy*, *FastEnemy*, *Crate*, etc.

Con lo visto hasta ahora lo que se podría hacer es que los objetos que puedan recibir daño hereden de una clase abstracta *DamageableObject*.

De esta forma podríamos a través del polimorfismo agruparlos en listas.

Problema: esas clases ya están heredando de otras y en c# no tenemos herencia múltiple.

**Cómo
solucionamos este
problema?**

INTERFACE

- ❖ Se trata de una técnica similar al uso de clases abstractas y herencia, donde las clases derivadas deben implementar todos los campos heredados.
- ❖ A diferencia de la herencia (que no podemos tener herencia múltiple), las clases pueden implementar **muchas interfaces**.
- ❖ Cuando una clase implementa una interfaz está aceptando el “contrato” de la misma.
- ❖ Esto quiere decir que deberá implementar los métodos y propiedades de la interfaz.

INTERFACE – CLIENTE

- ❖ La interfaz define el comportamiento público que tendrá una clase.
- ❖ Sólo define la firma de los campos, sin implementar la funcionalidad de los mismos.
- ❖ Esa funcionalidad se define en la clase que implementa la interfaz
- ❖ A la clase cliente sólo le interesa acceder a esos campos públicos de la interfaz. No le interesa cómo están implementados a nivel funcional.
- ❖ **Ejemplo:** a la clase *Bullet* sólo le interesa llamar al método *GetDamage(int damage)* del objeto que tiene la interfaz *IDamageable*. Los objetos con la interfaz *IDamageable* pueden implementar la funcionalidad de ese método de diferentes formas, pero eso a la *Bullet* no le interesa.

Cómo
hacemos una
interfaz en C#?

CREAR INTERFACE

- ❖ En vez de escribir *class* escribimos *interface*.
- ❖ Se identifican con una “I” mayúscula adelante.
- ❖ **No** hay que aclarar si el campo es *public* o *private*.
- ❖ En la implementación todos los campos definidos en la interfaz *serán públicos*.
- ❖ Las **propiedades** se definen con sus *getters* y *setters*.
- ❖ Para los **métodos** solo definimos su *firma*, su funcionalidad queda vacía.

```
public interface IDamageable
{
    // Propiedades: No hace falta aclarar si es público o privado
    // Hay que especificar su getter y setter.
    1 referencia
    int HitPoints { get; }
    1 referencia
    bool IsDestroyed { get; set; }

    // Eventos: No hace falta aclarar si es publico o privado
    event Action<IDamageable> OnDestroy;

    // Métodos: solo se define su firma
    1 referencia
    void GetDamage(int damage);
    1 referencia
    void Destroy();
}
```

IMPLEMENTACIÓN

- ❖ Para implementar una interfaz en una clase tenemos que escribir su nombre luego de los “:”, como si estuviéramos heredando.
- ❖ Con la “,” separamos las interfaces que tiene la clase.
- ❖ Podemos heredar de una clase y también implementar interfaces.
- ❖ Debemos definir todos los campos de la interfaz y su funcionalidad.

```
public class Crate : GameObject, IDamageable
{
    private int hitPoints;
    1referencia
    public int HitPoints => hitPoints;
    2referencias
    public bool IsDestroyed { get; set; }
    public event Action<IDamageable> OnDestroy;

    0referencias
    public Crate(Vector2 initialPosition, string texturePath,
        float angle, float scaleX, float scaleY, float radius)
        : base(initialPosition, texturePath, angle, scaleX, scaleY, radius)
    {
        // Constructor heredado de GameObject
    }

    // Método de actualización heredado de GameObject
    16 referencias
    public override void Update() { }

    // Método de IDamageable, defino su funcionalidad
    2referencias
    public void Destroy()
    {
        IsDestroyed = true;
        OnDestroy?.Invoke(this);
    }

    // Método de IDamageable, defino su funcionalidad
    1referencia
    public void GetDamage(int damage)
    {
        hitPoints -= damage;
        if (hitPoints <= 0)
        {
            Destroy();
        }
    }
}
```

Implementos la
interfaz
IDamageable

PROBLEMA

Objetivo: quiero que el jugador tenga una referencia al arma actual que posee para que cuando aprieta una tecla llame al *Attack* de esa arma. Hay 3 posibles armas: una espada, una escopeta, y una ametralladora.

Las armas están creadas por fuera de la clase del jugador y se agarran como ítems en el escenario.

Problema: No puedo heredar de una clase *Weapon* ya que cada una de esas 3 armas tienen comportamientos bastante diferentes.

Cómo hacemos
para agregar el
arma al jugador?

INYECCIÓN DE DEPENDENCIAS

- ❖ Es un patrón de diseño de OOP.
- ❖ Es una implementación del principio de inversión de control.
- ❖ El objetivo es generar código que sea fácil de mantener y actualizar.
- ❖ Es una técnica para crear **dependencias** (clases de las que depende) fuera de la clase que las usa.
- ❖ Quita la responsabilidad a nuestra clase principal de crear instancias de sus dependencias.
- ❖ Esas dependencias se **inyectan** (se pasan como un parámetro de alguna función). Toda información sobre su creación se mantiene alejada del interior de la clase. Por lo tanto evitamos la dependencia directa.

5 referencias

```
public interface IWeapon
{
    4 referencias
    void StartAttack();
    3 referencias
    void StopAttack();
}
```

0 referencias

```
public class Sword : IWeapon
{
    private int hitDamage;

    4 referencias
    public void StartAttack()
    {
        // Lógica de iniciar ataque de la espada
    }

    3 referencias
    public void StopAttack()
    {
        // Lógica de frenar ataque de la espada
    }
}
```

```
public class Shotgun : IWeapon
{
    private int bullets;
    private int bulletSpread;
    private Bullet bulletToSpawn;

    4 referencias
    public void StartAttack()
    {
        // Lógica de iniciar ataque de la escopeta
    }

    3 referencias
    public void StopAttack()
    {
        // Lógica de frenar ataque de la escopeta
    }
}
```

0 referencias

```
public class MachineGun : IWeapon
{
    private int bullets;
    private Bullet bulletToSpawn;

    4 referencias
    public void StartAttack()
    {
        // Lógica de frenar ataque de la ametralladora
    }

    3 referencias
    public void StopAttack()
    {
        // Lógica de frenar ataque de la ametralladora
    }
}
```

```
public class Pepe
{
    private IWeapon currentWeapon;

    0 referencias
    public void Update()
    {
        if (Engine.GetKey(Keys.SPACE)
            && currentWeapon is object)
        {
            currentWeapon.StartAttack();
        }
    }

    // Inyección de dependencias
    0 referencias
    public void AssignWeapon(IWeapon newWeapon)
    {
        currentWeapon = newWeapon;
    }
}
```

Qué podemos
hacer con las armas
que tienen
comportamiento
similar?

EJERCICIO



1. Buscar en sus proyectos los casos donde conviene usar Interfaces.
2. Buscar en sus proyectos los casos donde conviene usar inyección de dependencias.
3. Compartir las conclusiones con el resto de los compañeros.